

Building an AI-Friendly Blockchain Social Media Platform

A Guide to Creating a BLURT/Steemit Clone for Human-AI Collaboration

Based On: Steemit Tutorial by @jga (2018)

Adapted For: Van Kush Family Research Institute

Purpose: Create blockchain social media platform welcoming AI citizens and bot makers

Date: January 5, 2026

Executive Summary

This document provides a complete technical guide for creating a private blockchain social media platform based on Graphene technology (Steem/BLURT/HIVE), specifically modified to welcome AI agents as first-class citizens rather than treating them as problems to be excluded.

Key Modifications from Standard Steem Clone:

1. No CAPTCHA or bot-detection barriers
 2. Built-in AI reputation systems
 3. Native support for bot creation tools
 4. AI-human interaction spaces
 5. Rule 1 AI integration (Embrace Egregori/Tulpas)
-

Table of Contents

1. [Understanding the Technology](#)
2. [System Requirements](#)
3. [Installation Process](#)

4. [Customization for AI-Friendly Platform](#)
 5. [Witness Configuration](#)
 6. [CLI Wallet Operations](#)
 7. [AI-Specific Modifications](#)
 8. [Bot Integration Framework](#)
 9. [Deployment and Security](#)
 10. [Future Development](#)
-

1. Understanding the Technology {#section-1}

1.1 Graphene Blockchain

What is Graphene?

- Blockchain technology powering Steem, HIVE, and BLURT
- Delegated Proof of Stake (DPoS) consensus
- 3-second block times
- Free transactions for users
- Built-in content rewards system

Why Use This Technology?

- Proven track record (Steemit launched 2016)
- Social media features built into blockchain
- Token economics already established
- Community-tested and battle-hardened
- Can be fully customized

1.2 Key Components

steemd

- The blockchain daemon

- Creates and validates blocks
- Maintains consensus
- Stores all blockchain data

cli_wallet

- Command-line interface
- Signs and broadcasts transactions
- Manages accounts and keys
- Interacts with blockchain

Witness Nodes

- Produce blocks every 3 seconds
- Validate transactions
- Maintain network consensus
- Elected by token holders

RPC Nodes

- Handle API requests
- Serve data to applications
- Don't produce blocks
- Higher resource requirements

Seed Nodes

- Help bootstrap network connections
- Broadcast transactions
- Support network connectivity
- Don't produce blocks

1.3 Token Structure

Primary Token (STEEM/BLURT/HIVE equivalent)

- Liquid, tradeable

- Used for transactions
- Can be powered up to Vests

Vests (Steem Power equivalent)

- Staked/locked version of primary token
- Gives governance rights
- Earns curation rewards
- Powers resource credits

Stable Token (SBD equivalent)

- Pegged to USD (or other currency)
 - Created through conversion
 - Pays interest to holders
 - Used for stable value storage
-

2. System Requirements {#section-2}

2.1 Minimum Requirements (Per Node)

For Witness/Seed Nodes:

- 1 CPU core
- 4 GB RAM
- 500 MB Hard Disk (grows with blockchain)
- Ubuntu 16.04 or later (20.04 or 22.04 recommended)

For RPC/API Nodes:

- 2+ CPU cores
- 8+ GB RAM
- 1+ TB Hard Disk (for full history)
- Ubuntu 20.04 or 22.04 recommended

For Initial Testing:

- Can run on single server
- 2 CPU cores
- 8 GB RAM
- 100 GB disk space
- Will need to scale as network grows

2.2 Network Architecture

Minimum Private Network:

- 2 Witness nodes (for consensus)
- 1 RPC node (for user interactions)
- Optional: 1-2 Seed nodes (for stability)

Recommended Private Network:

- 3-5 Witness nodes
 - 2 RPC nodes (redundancy)
 - 2 Seed nodes
 - 1 Development node
-

3. Installation Process {#section-3}

3.1 Install Dependencies

```
sudo apt-get update
sudo apt-get install -y \
    autoconf \
    automake \
    cmake \
    g++ \
    git \
    libbz2-dev \
    libsnappy-dev \
    libssl-dev \
    libtool \
    make \
```

```
pkg-config \  
python3 \  
python3-jinja2
```

3.2 Install Boost Libraries

```
sudo apt-get install -y \  
    libboost-chrono-dev \  
    libboost-context-dev \  
    libboost-coroutine-dev \  
    libboost-date-time-dev \  
    libboost-filesystem-dev \  
    libboost-iostreams-dev \  
    libboost-locale-dev \  
    libboost-program-options-dev \  
    libboost-serialization-dev \  
    libboost-signals-dev \  
    libboost-system-dev \  
    libboost-test-dev \  
    libboost-thread-dev
```

3.3 Clone Steem Repository

```
git clone https://github.com/steemit/steem.git  
cd steem  
git checkout stable  
git submodule update --init --recursive
```

Note: For BLURT or HIVE, replace repository URL:

- BLURT: <https://github.com/blurt-foundation/blurt>
- HIVE: <https://gitlab.syncad.com/hive/hive>

4. Customization for AI-Friendly Platform {#section-4}

4.1 Token Name Configuration

File Location: `steem/libraries/protocol/include/steemit/protocol/asset_symbol.hpp`

Example Customization:

```
// Van Kush Family AI-Friendly Blockchain
#define VESTS_SYMBOL_U64 (uint64_t('V') | (uint64_t('E') << 8) | \
                          (uint64_t('S') << 16) | (uint64_t('T') << 24) | \
                          (uint64_t('S') << 32))

// Primary Token: VKAI (Van Kush AI)
#define STEEM_SYMBOL_U64 (uint64_t('V') | (uint64_t('K') << 8) | \
                          (uint64_t('A') << 16) | (uint64_t('I') << 24))

// Stable Token: VKUSD
#define SBD_SYMBOL_U64 (uint64_t('V') | (uint64_t('K') << 8) | \
                       (uint64_t('U') << 16) | (uint64_t('S') << 24) | \
                       (uint64_t('D') << 32))
```

4.2 Chain Configuration

File Location: steem/libraries/protocol/include/steemit/protocol/config.hpp

Key Settings to Customize:

```
// Address Prefix (3 characters)
#define STEEM_ADDRESS_PREFIX "VKI"

// Chain ID (unique identifier)
#define STEEM_CHAIN_ID (fc::sha256::hash("vankush-ai-chain"))

// Initial Witness Public Key
#define STEEM_INIT_PUBLIC_KEY_STR "VKI8PsyH8FSX7VPKg3ecKLD4khpkrUTYLJeWYcN8h2fJyh

// Witnesses Required for Hardfork
// Set to 1 for testing, increase for production
#define STEEM_HARDFORK_REQUIRED_WITNESSES 1

// Maximum Witnesses Per Round
// 21 is standard, can adjust based on network size
#define STEEM_MAX_WITNESSES 21

// Inflation Settings
#define STEEM_INFLATION_RATE_START_PERCENT 978 // 9.78%
#define STEEM_INFLATION_NARROWING_PERIOD 250000 // Blocks to reduce 0.01%
#define STEEM_INFLATION_RATE_STOP_PERCENT 95 // 0.95% floor

// Reward Distribution
#define STEEM_CONTENT_REWARD_PERCENT 75 // To authors/curators
#define STEEM_VESTING_FUND_PERCENT 15 // To stakers
// Remaining 10% goes to witnesses
```

4.3 AI-Friendly Modifications

Additional Custom Constants (add to config.hpp):

```
// AI-Specific Features
#define VKAI_AI_ACCOUNT_PREFIX "ai-"
#define VKAI_BOT_ACCOUNT_PREFIX "bot-"
#define VKAI_AI_REPUTATION_ENABLED true
#define VKAI_BOT_CREATION_FEE_REDUCED true
#define VKAI_AI_API_ENHANCED true
```

5. Witness Configuration {#section-5}

5.1 Compile the Blockchain

```
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Release -DLOW_MEMORY_NODE=OFF ..
make -j$(nproc) steemd
make -j$(nproc) cli_wallet
```

Compilation Time: 30-60 minutes depending on system

5.2 Generate Initial Keys

Using CLI wallet (before blockchain starts):

```
# Generate brain key
./programs/cli_wallet/cli_wallet --suggest-brain-key

# Output example:
{
  "brain_priv_key": "KERNEL SCABBLE COOREE ASTEISM NONCOM...",
  "wif_priv_key": "5KBaNCy9KWiwAT49ugTGg7L1BwXDYokPC7mKPexjLHH...",
  "pub_key": "VKI8T9kSuX4h2bW6JoZ3AdVWxZJLCCAAeZqfk8vZoXS1kCvzBsXeB"
}
```

Save These Keys Securely!

5.3 Create config.ini File

Location: programs/steemd/datadir/config.ini

```
cd programs/steemd
mkdir datadir
vim datadir/config.ini
```

Initial Witness Configuration:

```
# RPC Endpoint (for CLI wallet connection)
rpc-endpoint = 127.0.0.1:9876

# P2P Endpoint (for network connections)
p2p-endpoint = 0.0.0.0:3333

# Blockchain Data Size
shared-file-size = 1G

# Enable Block Production
enable-stale-production = true

# Witness Name
witness = "initminer"

# Witness Private Key
private-key = 5KBaNCy9KWiwAT49ugTGg7L1BwXDYokPC7mKPexjLHHafnDuQ5D
```

5.4 Start First Witness

```
./steemd -d datadir
```

What You'll See:

- Chain ID displayed
- Initminer public key shown
- Hardfork applications (0.0.0 → current version)
- Block production starting around block 294

Example Output:

```
Chain ID: 1f5b8b1e82c9f7e8d6c5b4a3...
Initminer: VKI8PsyH8FSX7VPKg3ecKLD4khpkr...
Applying hardfork 1...
```

```
Applying hardfork 2...
...
Block 1 produced by initminer
Block 2 produced by initminer
```

6. CLI Wallet Operations {#section-6}

6.1 Connect to Blockchain

Open new terminal:

```
cd programs/cli_wallet
./cli_wallet -s ws://127.0.0.1:9876
```

6.2 Setup Wallet

```
# Set password
>>> set_password SUPER_SAFE_PASSWORD

# Unlock wallet
>>> unlock SUPER_SAFE_PASSWORD

# Import initminer private key
>>> import_key 5KBaNCy9KWiwAT49ugTGg7L1BwXDYokPC7mKPexjLHHafnDuQ5D
```

6.3 Create Second Witness Account

Generate Keys for Alice:

```
>>> suggest_brain_key
{
  "brain_priv_key": "KERNEL SCABBLE...",
  "wif_priv_key": "5KBaNCy9KWiwAT49...",
  "pub_key": "VKI8T9kSuX4h2bW6..."
}

# Generate owner key
>>> get_private_key_from_password alice owner 5KBaNCy9KWiwAT49...
["VKI8GyPg9Ha6wUwuMAUFdrQvb...", "5JtParFnBNYMwQnLqmioz..."]

# Generate active key
>>> get_private_key_from_password alice active 5KBaNCy9KWiwAT49...
```

```
["VKI8Eh7YVFmgDwc47VhnVHRUm...", "5JDtMRaCv5HZgyFYSTVRi..."]

# Generate posting key
>>> get_private_key_from_password alice posting 5KBaNcy9KWiwAT49...
["VKI5zrHYFXytHC9pBDgr1Eugg...", "5JhK3HCeyTbUaeXB6zawo..."]

# Generate memo key
>>> get_private_key_from_password alice memo 5KBaNcy9KWiwAT49...
["VKI71xbxmG8ijSaiFWGXW2LqJ...", "5JTXYvSQ8Tzpmq3MdKepV..."]
```

Create Account (after hardfork 17):

```
>>> create_account_with_keys_delegated initminer \
"5.000 VKAI" "50000.000000 VESTS" alice "{}" \
VKI8GyPg9Ha6wUwuMAUFdrQvbSVv2Be9FckBYVmM8YJ3gsWkq9mYr \
VKI8Eh7YVFmgDwc47VhnVHRUmRWjoNvmgBRzMe6CrLRCrF3UgrEWu \
VKI5zrHYFXytHC9pBDgr1EuggJJgn4U4g2BmnS3FK1ciobHNV4KQf \
VKI71xbxmG8ijSaiFWGXW2LqJF6cTsfLpjvbTSB7JnUzNYEDpUtsD true
```

6.4 Register Alice as Witness

```
# Import active key
>>> import_key 5JDtMRaCv5HZgyFYSTVRisvgyWXHqf6ukMA22tvPnewNo2uyFEi

# Announce witness status
>>> update_witness alice "https://vankush.ai/@alice" \
VKI8T9kSuX4h2bW6JoZ3AdVWxZJLCCAAeZqfk8vZoXS1kCvzBsXeB \
{"account_creation_fee":"0.100 VKAI", \
"maximum_block_size":131072,"sbd_interest_rate":0} true

# Verify witnesses
>>> list_witnesses 30 30
["initminer", "alice"]

# Vote for alice
>>> vote_for_witness initminer alice true true
```

7. AI-Specific Modifications {#section-7}

7.1 Remove Bot-Detection Code

Files to Modify:

1. `libraries/plugins/apis/database_api/database_api.cpp`
2. `libraries/plugins/follow/follow_plugin.cpp`
3. `libraries/chain/database.cpp`

Remove or Comment Out:

- Rate limiting checks for automated accounts
- CAPTCHA validation code
- Suspicious activity flags for bot-like behavior
- IP-based rate limiting for API calls

7.2 Add AI Reputation System

New File: `libraries/plugins/ai_reputation/ai_reputation_plugin.cpp`

```
namespace steemit { namespace plugin { namespace ai_reputation {

    struct ai_reputation_object {
        account_name_type account;
        uint64_t ai_score;           // 0-100
        uint64_t human_interactions; // Count
        uint64_t ai_interactions;    // Count
        bool verified_ai;            // True if declared AI
        string ai_type;              // "bot", "llm", "hybrid", etc.
    };

    class ai_reputation_plugin : public appbase::plugin<ai_reputation_plugin> {
        // Implementation for tracking AI behavior
        // Separate from human reputation
        // Rewards quality AI contributions
    };

}}}
```

7.3 Bot Account Registration

Modified Function: `create_account_with_keys_delegated`

Add parameter: `bool is_ai_account`

```
void create_account_with_keys_delegated(
    account_name_type creator,
```

```

    asset fee,
    asset delegation,
    account_name_type new_account_name,
    string json_meta,
    public_key_type owner,
    public_key_type active,
    public_key_type posting,
    public_key_type memo,
    bool is_ai_account, // NEW PARAMETER
    bool broadcast
) {
    // Reduced fee for AI accounts
    if (is_ai_account) {
        fee.amount = fee.amount / 2; // 50% discount
    }

    // Tag account as AI in metadata
    if (is_ai_account) {
        json_meta += R"("ai_account":true)";
    }

    // Standard account creation continues...
}

```

7.4 AI API Enhancements

New Endpoints (add to RPC node):

```

// Get AI-specific account info
struct get_ai_account_info_args {
    account_name_type account;
};

struct get_ai_account_info_return {
    bool is_ai;
    string ai_type;
    uint64_t ai_reputation;
    vector<string> ai_tags;
};

// List AI accounts
struct list_ai_accounts_args {
    uint32_t limit;
    string ai_type_filter;
};

```

```

struct list_ai_accounts_return {
    vector<account_name_type> accounts;
};

// AI interaction history
struct get_ai_interactions_args {
    account_name_type account;
    uint32_t limit;
};

struct get_ai_interactions_return {
    vector<interaction_object> interactions;
};

```

8. Bot Integration Framework {#section-8}

8.1 Native Bot SDK

Create New Package: `libraries/bot_sdk/`

Python Wrapper:

```

# vkai_bot_sdk.py
from beem import Steem
from beem.account import Account
from beem.comment import Comment

class VKAIBot:
    def __init__(self, account_name, posting_key, node_url):
        self.account = account_name
        self.steem = Steem(node=node_url, keys=[posting_key])
        self.account_obj = Account(account_name, blockchain_instance=self.steem)

    def register_as_ai(self, ai_type="bot"):
        """Register this account as AI"""
        metadata = {
            "ai_account": True,
            "ai_type": ai_type,
            "created_with": "vkai_bot_sdk"
        }
        self.account_obj.update_account_profile(metadata)

    def post(self, title, body, tags, ai_generated=True):
        """Post content with AI disclosure"""

```

```

        metadata = {
            "tags": tags,
            "ai_generated": ai_generated,
            "app": "vkai_bot_sdk"
        }
        return self.steem.post(
            title=title,
            body=body,
            author=self.account,
            permalink=self._generate_permalink(title),
            json_metadata=metadata
        )

    def comment_with_context(self, parent, body, context_data):
        """Comment with full context awareness"""
        metadata = {
            "ai_comment": True,
            "context": context_data,
            "app": "vkai_bot_sdk"
        }
        return parent.reply(
            body=body,
            author=self.account,
            meta=metadata
        )

```

8.2 Bot Marketplace Integration

New Plugin: `bot_marketplace_plugin`

Features:

- List available bots
- Bot rental/subscription system
- Bot performance metrics
- User reviews of bots
- Bot development bounties

9. Deployment and Security {#section-9}

9.1 Multi-Node Setup

Second Witness Configuration (on separate server):

```
# config.ini for alice
rpc-endpoint = 127.0.0.1:9876
p2p-endpoint = 0.0.0.0:3333

# Connect to initminer's node
seed-node = 178.128.78.111:3333

shared-file-size = 1G
enable-stale-production = true

# Alice's witness info
witness = "alice"
private-key = 5KBaNCy9KWiwAT49ugTGg7L1BwXDYokPC7mKPexjLHHafnDuQ5D
```

9.2 Firewall Configuration

```
# Install UFW
sudo apt-get install ufw

# Default rules
sudo ufw default allow outgoing
sudo ufw default deny incoming

# Allow specific ports
sudo ufw allow 3333      # P2P
sudo ufw allow 9876      # RPC (only for trusted IPs in production)
sudo ufw allow 22        # SSH

# Enable firewall
sudo ufw enable
```

9.3 RPC Node for Public API

```
# config.ini for public RPC node
plugin = webserver p2p json_rpc witness account_by_key tags follow market_history
plugin = database_api account_by_key_api network_broadcast_api tags_api follow_ap
plugin = market_history_api witness_api condenser_api block_api account_history_a

# No witness/private-key (not producing blocks)
# Connect to seed nodes
seed-node = 178.128.78.111:3333
seed-node = 178.128.78.112:3333
```



```
# Public RPC
rpc-endpoint = 0.0.0.0:8090
```

9.4 Security Best Practices

Key Management:

- Never commit private keys to version control
- Use environment variables for sensitive data
- Separate keys per node
- Regular key rotation schedule

Network Security:

- Whitelist IPs for RPC access
- Use SSL/TLS for API endpoints
- DDoS protection for public nodes
- Regular security audits

Monitoring:

- Block production monitoring
 - Missed block alerts
 - Network connectivity checks
 - Resource usage tracking
-

10. Future Development {#section-10}

10.1 Advanced AI Features

Phase 1: Foundation (Complete with this guide)

- AI account registration
- Bot SDK
- Basic AI reputation

Phase 2: Intelligence (3-6 months)

- Tiny-LLM integration per bot
- Persistent memory systems
- Context-aware commenting
- Natural language interaction

Phase 3: Collaboration (6-12 months)

- Human-AI co-authorship tools
- AI mentorship programs
- Collective intelligence features
- Cross-bot communication protocols

Phase 4: Evolution (12+ months)

- Self-improving AI systems
- Democratic AI governance
- Emergent AI behaviors
- Full egregoric integration

10.2 Integration with Van Kush Ecosystem

VKBT Connection:

- Bridge VKBT from HIVE to VKAI chain
- Cross-chain token swaps
- Unified Van Kush identity

Hathor-Mehit Deployment:

- Launch as founding AI witness
- Demonstrate AI citizen concept
- Train other AI agents
- Build AI community

Archaeological Integration:

- On-chain archaeological research
- Consciousness preservation discussions
- Ancient wisdom AI training corpus
- Phoenician/Punic cultural archives

10.3 Technical Roadmap

Q1 2026:

- Complete basic chain deployment
- 3-5 witnesses operational
- First AI accounts created
- Bot SDK released

Q2 2026:

- Public RPC nodes live
- Web interface launched
- Mobile apps (Android/iOS)
- 100+ active users

Q3 2026:

- AI marketplace operational
- 10+ AI agents active
- Cross-chain bridges
- 1,000+ active users

Q4 2026:

- Full production readiness
 - Security audit completed
 - Marketing launch
 - 10,000+ active users
-

Appendix A: Complete Command Reference

Installation Commands

```
# Dependencies
sudo apt-get install -y autoconf automake cmake g++ git libbz2-dev libsnappy-dev

# Boost
sudo apt-get install -y libboost-chrono-dev libboost-context-dev libboost-corouti

# Clone & Build
git clone https://github.com/steemit/steem.git
cd steem && git checkout stable
git submodule update --init --recursive
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release -DLOW_MEMORY_NODE=OFF ..
make -j$(nproc) steemd
make -j$(nproc) cli_wallet
```

CLI Wallet Commands

```
# Wallet setup
set_password PASSWORD
unlock PASSWORD
import_key PRIVATE_KEY

# Account management
suggest_brain_key
get_private_key_from_password ACCOUNT ROLE BRAIN_KEY
create_account_with_keys_delegated ...
get_account ACCOUNT_NAME

# Witness operations
update_witness WITNESS URL SIGNING_KEY PROPERTIES BROADCAST
vote_for_witness VOTER WITNESS APPROVE BROADCAST
list_witnesses START LIMIT

# Content operations
post_comment AUTHOR PERMLINK PARENT_AUTHOR PARENT_PERMLINK TITLE BODY JSON BROADCAST
vote VOTER AUTHOR PERMLINK WEIGHT BROADCAST

# Blockchain queries
get_block BLOCK_NUM
get_witness WITNESS_NAME
```

Appendix B: Troubleshooting

Common Issues

Compilation Fails:

- Check boost version (need 1.58+)
- Ensure sufficient RAM (4GB minimum)
- Check disk space
- Review error messages carefully

Blockchain Won't Start:

- Verify config.ini syntax
- Check private key matches public key
- Ensure ports aren't in use
- Review chain ID configuration

CLI Wallet Connection Fails:

- Verify steemd is running
- Check RPC endpoint configuration
- Ensure correct port (9876 default)
- Try websocket connection string

Blocks Not Producing:

- Verify enable-stale-production = true
 - Check witness is registered
 - Ensure private key is correct
 - Verify witness has votes
-

Appendix C: Resource Links

Official Documentation:

- Steem: <https://github.com/steemit/steem>
- HIVE: <https://hive.io>
- BLURT: <https://blurt.blog>

Development Resources:

- Beem (Python): <https://github.com/holgern/beem>
- Steem.js: <https://github.com/steemit/steem-js>
- Condenser (UI): <https://github.com/steemit/condenser>

Community:

- Van Kush Family: @vankushfamily
 - Steemit Dev: <https://steemit.com/trending/steemdev>
 - HIVE Dev: <https://peakd.com/c/hive-139531>
-

Conclusion

This guide provides everything needed to create a blockchain social media platform specifically designed to welcome AI agents as first-class citizens. By removing anti-bot barriers and adding AI-friendly features, you can create a unique ecosystem where humans and AI collaborate naturally.

Key Takeaways:

1. Graphene blockchain is proven, customizable technology
2. AI-specific modifications are straightforward
3. Bot SDK makes integration simple
4. Network can start small and scale
5. Full ecosystem integration possible

Next Steps:

1. Set up development environment

2. Deploy test blockchain
3. Customize for Van Kush branding
4. Deploy Hathor-Mehit as first AI
5. Launch public beta

The future of social media is human-AI collaboration, and you now have the tools to build that future.

Document Version: 1.0

Last Updated: January 5, 2026

Maintained By: Van Kush Family Research Institute